

Interrupts&System-Calls - Zusammenfassung

Interrupt

Ein Interrupt (DE: Unterbrechung) ist ein Signal, welches aus verschiedenen Quellen, meistens Hardwarekomponenten, stammen kann.

Das Signal, welches gesendet wird, wird als Interrupt bzw. **Interrupt Request** bezeichnet.

Idee und Ablauf

Idee

Die Idee eines Interrupts ist es, bei einer wichtigen Änderung in einer Hardwarekomponente (zB. Bewegen der Maus, Eingabe auf der Tastatur) ein Signal an die CPU zu senden, auf welche diese dann mit reagieren kann.

Der große Vorteil ist, dass die Signale **von den Hardwarekomponenten aktiv gesendet werden**. Ohne Interrupts müsste die CPU aktiv immer wieder die Hardwarekomponente nach Änderungen fragen (Dieses Verhalten der CPU wird als **Polling** bezeichnet). Dies würde viel Zeit der CPU in Anspruch nehmen und auch sehr schnelle Eingaben ausbremsen, was zu einer merkbaren Verzögerung führen würde.

Ablauf - Asynchroner HW Interrupt "Mausbeispiel"

- zB. Benutzer bewegt die Maus
- Die Sensoren der Maus erkennen die Bewegung, übersetzen diese in Daten
- Diese Daten werden formatiert und als Interrupt Request an die CPU geschickt
- CPU erhält die Nachricht und stoppt aktuellen Prozess
- Für den Interrupt gibt es eine fertige Reaktion (Interrupt Handler) im Treiber der Maus
- Dieser wird in die CPU geladen und verarbeitet die Daten entsprechend
- Ist der Interrupt Handler fertig ausgeführt wird dieser wieder geschlossen und der normale Prozess wird wieder fortgesetzt

Wichtigste Punkte dieses Ablaufs aus Sicht der Interrupts:

- Auslösung auf HW
- Übertragung an CPU
- Auslösen des Interrupt Handlers
- Beenden des Interrupts

Interrupt Ablauf auf Hardwareebene

Auf Hardwareebene betrachtet passiert noch etwas mehr als oben beschrieben.

- Ein externes Gerät kommuniziert mit dem PC über eine Art Bus. Dieser ist mit dem Programmable Interrupt Controller (PIC) verbunden.
- Der PIC ist dafür da, alle ankommenden Interrupts zu sammeln und nach **Wichtigkeit** zu priorisieren. zB. Abort > Fault > Trap. Nach der Priorisierung lässt er den **nächstwichtigsten** passieren.
- Der PIC setzt ein Bit, welches quasi ein Flag für die CPU ist, dass anzeigt wenn ein Interrupt vorliegt.
- Dieses Bit wird von der CPU nach Abschluss jeder Instruktion geprüft.
- Ist es gesetzt, werden zuerst alle Zustände des derzeit laufenden Prozesses zwischengespeichert.
- Dann wird die Nummer des Interrupts ausgelesen und der passende Interrupt Handler gesucht
- Wird dieser gefunden, wird dieser in den Prozessor geladen und ausgeführt.
- Ist die ISR fertig, wird der vorherige Prozess wieder in den Prozessor geladen.

Interrupt Service Routine - Interrupt Handler

Der Interrupt Handler bzw. Die Interrupt Service Routine (ISR), ist ein Programm bzw. Programmteil, welcher im Treiber von Hardwaregeräten liegt. Er hat die Aufgabe, die zu ihm passenden Interrupts korrekt zu bearbeiten und aufzulösen.

Hat ein Gerät mehrere verschiedene Interrupts, gibt es mehrere Interrupt Handler, welche darauf reagieren. Interrupt Handler sollten generell möglichst leichtgewichtig und schnell in der Ausführung sein, um lange Bearbeitungszeiten auf der CPU zu vermeiden.

Systemspezifisches Interrupt Handling: Linux

Je nach Betriebssystem werden Interrupts auf unterschiedliche Weise erkannt und gespeichert. Unter Linux werden Interrupt-Nummern in einem eigenen Table gespeichert. Interrupt Descriptor Table (IDT). Der Speicherort der Tabelle ist in einem Register gespeichert.

Diese Tabelle beinhaltet die Nummern der Interrupts, damit das OS diese korrekt den ISR zuordnen kann. ISRs sind in 2 Teile aufgeteilt.

Der erste Teil, der wahre Interrupt Handler, muss wirklich schnell sein, da dieser Teil auch andere Interrupts blockiert, bis er verarbeitet wurde.

Der zweite Teil, welcher von anderen Interrupts wieder unterbrochen werden kann, ist für die weitere Verarbeitung des Interrupts da.

Interruptquellen

Interrupts können aus verschiedensten Quellen stammen. Meistens stammen sie aus anderen Hardwaregeräten. Allerdings gibt es auch intern Interrupts, welche auf verschiedene Umstände hinweisen.

Fehler und Ausnahmen

Beispielsweise werden oft Interrupts ausgelöst wenn innerhalb eines Prozesses ein Fehler / eine Ausnahme auftritt. Beispiele hierfür sind unter anderem Division durch Null, Adressierungsfehler ...

Nutzung eines Interrupts durch Prozess - System Calls

Ein Prozess kann gezielt einen Interrupt verwenden, um eine ISR auszulösen und dann das Ergebnis dieser im eigenen Prozess wieder weiterzuverwenden. Dies geschieht sehr oft:

- Lesen / Schreiben einer Datei - Prozess braucht Hilfe des OS / Dateisystems
- Datenbeschaffung aus dem Internet - Prozess braucht Zugriff und Hilfe des OS
- Semaphore
- Vieles mehr

Kurzerläuterung Semaphore

Semaphore sind ein Begriff aus dem Bereich des Multithreadings. Ohne tiefer ins Detail zu gehen: Laufen mehrere Threads "parallel" und tauschen dabei Daten aus, kann es zu Situationen kommen, welche die Datenkonsistenz zerstören. zB. Kann ein Prozess1 eine Variable auslesen, erhöhen und dann zurückschreiben wollen, und ein anderer Prozess2 will die erhöhte Variable wieder auslesen und multiplizieren.

Wird der Prozess1 nun unterbrochen nachdem er die Variable gelesen hat, aber bevor er sie zurückgeschrieben hat, liest Prozess2 den falschen Wert aus und multipliziert diesen. Das Programm läuft korrekt, aber durch das Threading entsteht ein Fehler. Diese Fehlerquelle wird als **Race Condition** bezeichnet.

Um diese **Race Condition** zu verhindern gibt es **Semaphore** - Diese sind quasi Markierungen in einem Programm, welche der CPU sagen, dass das Programm hier niemals unterbrochen werden darf. Dadurch kann sichergestellt werden, dass der Schreibprozess in Prozess immer ausgeführt wird bevor ein anderer Prozess dran kommt.

Einteilung und Beispiele

(Diese Einteilung stammt von INTEL, nicht alle Geräte teilen Interrupts auf die gleiche Weise ein)

Grundsätzlich lassen sich Interrupts in 2 verschiedene Gruppen einteilen.

1. Asynchron zum Prozessor
2. Synchron zum Prozessor

Asynchrone Interrupts

Asynchrone Interrupts sind im wesentlichen alle Interrupts welche **außerhalb des Computers** entstehen, sprich alle HW Interrupts wie eine Mausbewegung oder eine Tasteneingabe.

Da sie Asynchron sind, können sie **jederzeit** bei der CPU ankommen, unabhängig von ihrem Status. Sie werden in einem kleinen Cache abgelegt und warten dort darauf, dass die CPU alles für ihre Bearbeitung vorbereitet.

Synchrone Interrupts

Synchrone Interrupts sind jene Interrupts, die innerhalb des Computers entstehen. Sie werden von Prozessen ausgelöst, entweder als System Call oder als Reaktion auf einen Fehler / eine Ausnahme.

Die Fehler (Exceptions) werden hierbei nochmals nach ihrer Schwere in verschiedene Untergruppen unterteilt.

Exception: Trap

Der "schwächste" Fehlerfall ist die **Trap**. Wird eine Trap ausgelöst, wird ihre Interrupt Request von der verfügbaren ISR bearbeitet und dann wird **der Prozess, der die Trap ausgelöst hat, von der nächstfolgenden Instruktion weiter ausgeführt**.

Traps sind für "vorhergesehene" Situationen da, beispielsweise basieren Break Points und System Calls auf diesem Exceptiontyp. Traps sind **gewollte Ausnahmen**

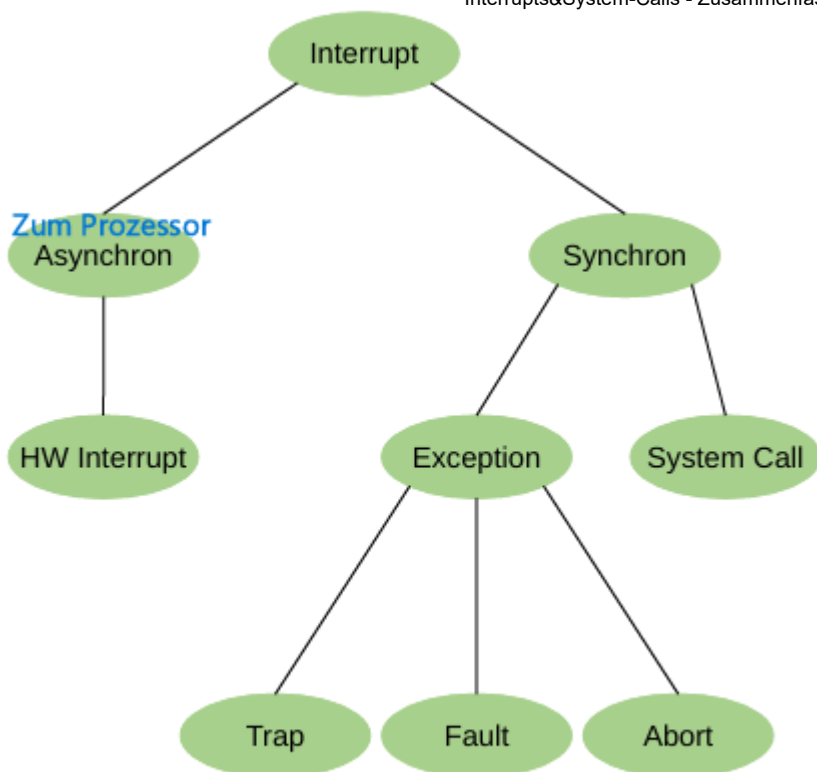
Exception: Fault

Ein Fault ist ein **ungewollter, aber behandelbarer Programmfehler**. Wird ein Fault ausgelöst, wird dieser wie jeder Interrupt von der ISR behandelt. Der Unterschied zur Trap ist, **dass der Befehl, welcher den Fault ausgelöst hat, wiederholt wird**. Beispielsweise bei einer Division durch Null könnte eine ISR sein, die 0 durch eine 1 zu ersetzen (unsinnig, aber möglich). Dann wird die Division erneut versucht und funktioniert nun.

Andere Beispiele könnten ein kurzes Problem mit der Internetverbindung sein, wo ein nicht erhaltenes Datenpaket erneut angefordert wird anstatt es zu übergehen oder den gesamten Download abubrechen.

Exception: Abort

Aborts sind **gravierende, nicht behandelbare Fehler, welche zu einem Prozessabsturz führen**. Diese treten auf, wenn etwas grundlegendes falsch gelaufen ist, zB. ein Fehler in der Hardware, korrupte Speichertabellen oder ähnliches. Erhält ein System einen Abort, fährt es sich selber herunter um noch größere Schäden zu vermeiden. **Hier ist die ISR also der System Shutdown**



Beispiel Synchroner Timer Interrupt

Ein Beispiel für einen synchronen Interrupt ist der PIT, der *Programmable Interval Timer*. Dieser ist dafür zuständig eine gleichbleibende periodische Intervalle zu liefern, um verschiedene zeitabhängige Funktionen zu ermöglichen.

Interrupt des PIT

- Periodischer Interrupt, alle x ms (programmierbar, meist alle 10ms ein Interrupt)
- Interrupt hat eine hohe Priorität, das heißt dass er sofort in die CPU geladen wird sofern nicht gerade ein anderer Interrupt aktiv abgearbeitet wird.

Interrupt Handler des PIT

- Aktualisiert die Uptime für jeden Interrupt
- Aktualisiert die Systemzeit
- Überprüft, ob der aktuelle Prozess seine maximal erlaubte CPU Zeit bereits überschritten hat.
- Aktualisiert Ressourcenstatistiken

System Calls

System Calls sind ein Spezialfall eines Interrupts (genauer gesagt einer Trap) welcher die Kommunikation zwischen dem OS / der Hardware und dem Nutzer übernimmt. Sie ermöglichen es Prozessen, über das OS auf die Hardware zuzugreifen.

Hierbei entscheidet das OS über die Validität der Anfragen der Prozesse und stoppt nicht konsistente Anfragen automatisch.

Interrupts & System Calls | Zusammenfassung

System Calls sind eine **Abstraktionsebene** der komplizierten Hardwarebefehle welche durch die Systemcalls aufgerufen werden und erleichtern v.a den Umgang mit Hardwarekomponenten wie Speicher und anderen Endgeräten. Sie verallgemeinern die Hardwarespezifischen Befehle und ermöglichen so die Nutzung vieler verschiedener Komponenten über das selbe OS.

User Space und Kernel Space

Wenn ein Nutzer etwas auf einem Computer macht, hat er standardmäßig nicht alle Berechtigungen. Das OS erlaubt nicht alles, zB Zugriff auf wichtige Ordner und Dateien, das Löschen von Systemfiles, oder auch das Umstellen mancher Einstellungen.

Diese Begrenzungen rühren aus einer Trennung zwischen dem OS und dem User in Spaces.

Über System Calls werden Befehle aus dem User Space in den Kernel Space gesendet und können vom OS verarbeitet werden.

User Space

Der User Space ist der Space in welchem alle Prozesse die nicht durch das OS gestartet werden standardmäßig liegen. Auch der User arbeitet im User Space, solange er keine SuperUser (Windows: Administrator) Rechte hat. Funktionen sind eingeschränkt, dafür schaut das OS dass man nicht versehentlich irreversible Schäden anrichtet oder Malware das System komprimiert.

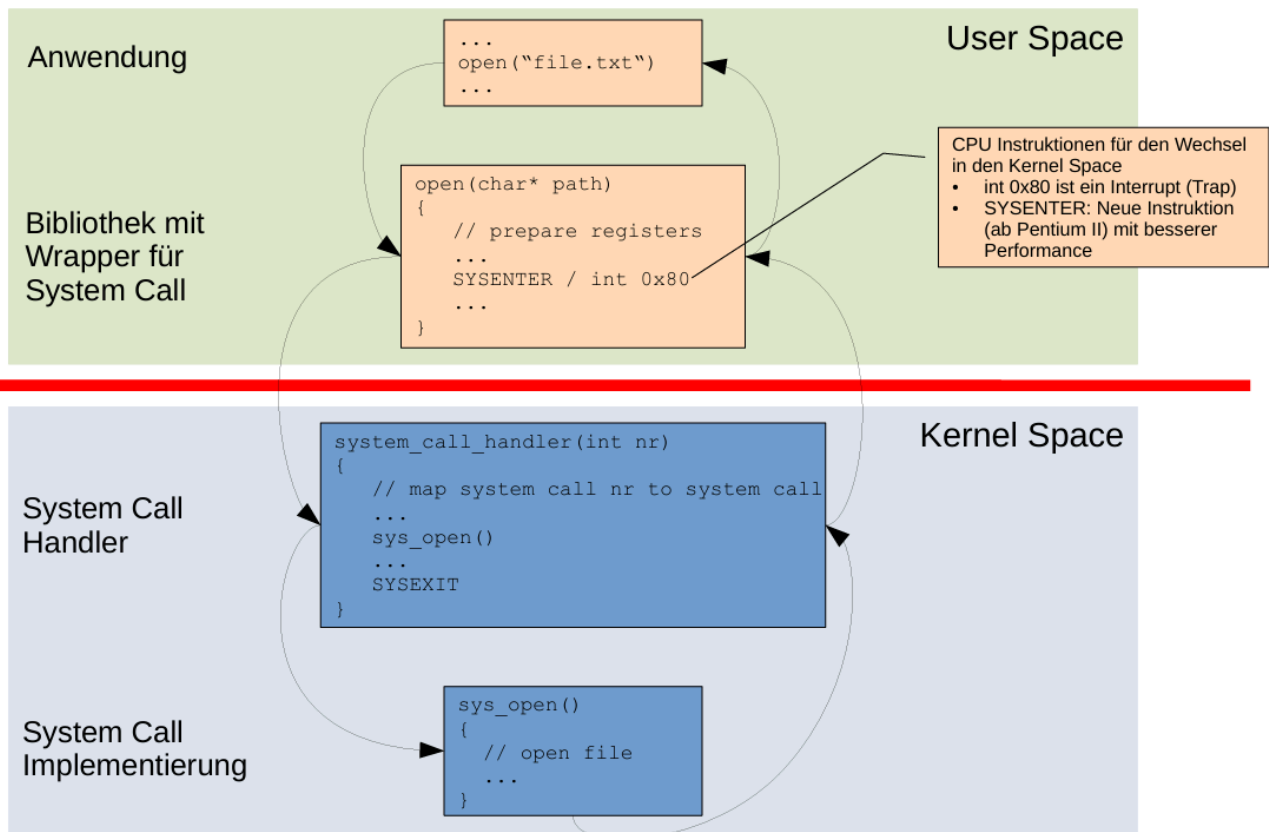
Kernel Space

Das OS und der SuperUser bzw. Administrator arbeiten im Kernel Space. Dieser ist quasi uneingeschränkt und lässt jegliche Aktion auf die Hardware zu. Das bedeutet natürlich ein erhebliches Sicherheitsrisiko, weswegen Prozesse nur durch ausdrückliche Einwilligung des Users in diesem Modus gestartet werden können. Das OS läuft standardmäßig im Kernels Space und verwaltet ihn.

Ablauf

Zuerst löst ein Prozess oder der User selbst einen System Call (Trap Interrupt) aus. Dafür wird eine vorgefertigte Funktion aufgerufen, welche intern den System Call absetzt. Dieser wird vom OS erkannt.

Das OS erkennt den Interrupt, macht die CPU für den passenden ISR frei und startet die Service



System Calls in Linux

Verschiedene OS verwenden verschiedene Systemcalls. Das ist auch der Grund, warum eine Windows EXE nicht auf einem Linux laufen kann, da Linux die Windowsbefehle nicht erkennt.

Linux unterstützt ca. 500 System Calls. Fast alle System Calls werden in **Wrappers** (kleine Funktionen) aufgerufen, welche neben dem Call selbst noch einige eigene kleine Aufgaben erfüllen:

- Funktionsargumente und die Nummer des System Calls in die richtigen Register kopieren (wie vom Kernel erwartet)
 - In den Kernel Mode wechseln (den Trap ausführen / den System Call senden)
 - Abholen des Return Codes und Rückgabe an den Aufrufer
- Die Namen von Wrapper und SysCall sind meist ident, immerhin haben sie auch nur die Aufgabe diesen spezifischen System Call zu facilitaten.

Beispiel - Mausbewegung auslesen über System Calls

Selbes Beispiel wie asynchroner Interrupt:



- + zB. Benutzer bewegt die Maus
- + Die Sensoren der Maus erkennen die Bewegung, übersetzen diese in Daten
- + Diese Daten werden formatiert und als Interrupt Request an die CPU geschickt
- + CPU erhält die Nachricht und stoppt aktuellen Prozess
- + Für den Interrupt gibt es eine fertige Reaktion (Interrupt Handler) im Treiber der Maus

+ Dieser wird in die CPU geladen und verarbeitet die Daten entsprechend

- Beispielerweiterung

Der Handler schreibt die aktuelle Mausposition in ein spezifisches File

Prozesse rufen über Systemcalls (read / open) dieses File auf und lesen die Werte aus

+ Ist der Interrupt Handler fertig ausgeführt wird dieser wieder geschlossen und der normale Prozess wird wieder fortgesetzt